Aspectos destacados

De la detección a la acción: cerrando el bucle autonómico de una línea de fabricación autocoordinada con planificadores basados en reglas y en LLM con recuperación aumentada

Manuel Sánchez, Ernesto Exposito

- La arquitectura autonómica de 2020 detectaba condiciones, pero nunca planificaba ni actuaba.
- Conectar las alarmas heredadas a reparaciones no aporta nada y repara estaciones sanas.
- Un test clásico de detección adaptativa de cambios es un baseline sólido para el triaje.
- Un LLM de frontera con conocimiento recuperado iguala ese test, pero no lo supera.
- Como veto sobre alarmas agrupadas, el LLM de frontera se vuelve demasiado conservador.

De la detección a la acción: cerrando el bucle autonómico de una línea de fabricación autocoordinada con planificadores basados en reglas y en LLM con recuperación aumentada

Manuel Sánchez[a,b,*], Ernesto Exposito[b]

[a] *Universidad Nacional Experimental del Táchira, San Cristóbal, Venezuela*
[b] *Univ Pau & Pays Adour / E2S UPPA, Laboratoire d'informatique de l'Université de Pau et des Pays de l'Adour (LIUPPA), EA3000, 64000 Pau, Francia*

Resumen

Las arquitecturas autonómicas para la Industria 4.0 prometen una producción que se autoconfigura, se autorrepara y se autooptimiza, pero los sistemas publicados a menudo se quedan en la detección. Revisamos nuestra propia arquitectura de fabricación autocoordinada de 2020 y su simulador, y comprobamos que implementa las fases Monitor y Analyze, pero no Plan, Execute ni una base de conocimiento: las condiciones detectadas solo producen un mensaje en un registro. Portamos el simulador a un motor sin interfaz gráfica y con pruebas, sobre el conjunto de datos de la línea de producción de Bosch, y cerramos el bucle con un espacio de acciones construido sobre las estaciones gemelas paralelas de la línea, un validador simbólico, inyección de fallos (escalón, intermitentes y deriva) y un modelo de efectos explícito. Comparamos planificadores basados en reglas con planificadores basados en modelos de lenguaje de gran tamaño (LLM), con y sin conocimiento operativo recuperado, sobre un modelo local de 7B (Qwen2.5) y un modelo de frontera (Claude Fable 5.1), usando una traza de desarrollo para el diseño y tres trazas reservadas para la evaluación. Conectar las alarmas originales directamente a reparaciones no aporta nada en las trazas reservadas (recuperación agregada de +0,02) y repara 87 estaciones sanas por escenario con fallo. El cuello de botella es el triaje de alarmas, no la planificación: un test clásico de detección adaptativa de cambios recupera entre el 52 y el 54 % del

*Autor de correspondencia.
Email addresses: mbsanchez@unet.edu.ve (Manuel Sánchez), ernesto.exposito@univ-pau.fr (Ernesto Exposito)

Versión de lectura en español. El manuscrito que se envía a Computers in Industry es la versión en inglés.

retraso causado por los fallos, y reajustar sus hiperparámetros en la traza de desarrollo no lo mejora. Con el modelo de 7B, el planificador con recuperación aumentada recupera el 20 %; con el modelo de frontera, el mismo diseño recupera el 58 %, estadísticamente indistinguible de la mejor regla (diferencia de +0,04, intervalo al 95 % de −0,01 a +0,10); sin conocimiento recuperado, el modelo de frontera se niega a actuar. En los fallos intermitentes queda por delante de todas las configuraciones de reglas que probamos, pero este resultado exploratorio se apoya en 24 escenarios de tres trazas y su intervalo llega al cero. Usado en cambio como veto sobre el detector original, el modelo de frontera se vuelve extremadamente conservador y atribuye casi todas las alarmas a una causa común de la línea, una estructura que el detector impone a todas sus alarmas. Cerrar bucles autonómicos en la fabricación depende, por tanto, más de la interfaz Analyze–Knowledge que de la capacidad del planificador.

Keywords: Computación autonómica, MAPE-K, Industria 4.0, Modelos de lenguaje de gran tamaño, Generación aumentada por recuperación, Gestión de fallos

---

1. Introducción

La computación autonómica propone sistemas que se gestionan a sí mismos mediante un bucle Monitor–Analyze–Plan–Execute sobre una base de conocimiento compartida (MAPE-K) [1, 2]. En la fabricación este bucle resulta atractivo porque las líneas de producción se degradan continuamente y la supervisión humana no escala: una línea que se autorrepara detectaría una estación que se degrada y se reconfiguraría antes de que el retraso se acumule.

En 2020 propusimos una arquitectura para procesos de fabricación autocoordinados con propiedades self-* y la evaluamos con un simulador, SSAC-Simulator, sobre el conjunto de datos de la línea de producción de Bosch [3, 4]. Al revisar ese código seis años después, comprobamos que solo implementa las fases Monitor y Analyze. Cuando uno de sus tres detectores se dispara, el simulador escribe *"launching the autonomous cycle for reconfiguration"* en un panel de registro; no se produce ningún plan y no se ejecuta nada. El bucle se anunciaba, pero no se cerraba.

Este artículo lo cierra y se pregunta qué hace falta realmente para cerrarlo. Dos desarrollos hacen oportuna la pregunta. Primero, los modelos de lenguaje de gran tamaño (LLM) se proponen cada vez más como planificadores para la toma de decisiones industriales, desde el control de sistemas

de producción modulares [5] hasta la planificación y programación avanzadas [6]. Segundo, la generación aumentada por recuperación (RAG) [7] es una candidata natural para el componente Knowledge de MAPE-K: en lugar de una ontología construida a mano, el planificador recupera el historial operativo que necesita. Si alguno de los dos ayuda, y frente a qué baseline, es una cuestión empírica.

Nuestras contribuciones son:

1. Un port fiel, sin interfaz gráfica y con pruebas del simulador de 2020, y un análisis de su capa Analyze sobre los datos publicados: uno de los tres detectores es inalcanzable, otro se ejercita con el 0,004 % de los eventos y el tercero se dispara de forma rutinaria en una traza sin fallos (sección 3).
2. Las fases Plan y Execute que faltaban: un espacio de acciones basado en las estaciones "gemelas" paralelas de la línea de Bosch, un validador simbólico, inyección de fallos con tres formas y un modelo de efectos explícito y parametrizado cuyos supuestos no verificables se barren en lugar de fijarse (sección 4).
3. Una comparación de planificadores basados en reglas con planificadores LLM con y sin conocimiento recuperado, y con un híbrido en el que un test estadístico hace el triaje y el LLM veta, sobre un modelo local de 7B y un modelo de frontera, con un protocolo que separa una traza de desarrollo de tres trazas reservadas (secciones 5–7).
4. Evidencia de que el cuello de botella para cerrar el bucle es el triaje de alarmas y no la planificación; de que, para el triaje con recuperación aumentada, un modelo de frontera iguala a un test estadístico bien ajustado donde un modelo de 7B no lo consigue, y el conocimiento recuperado es lo que le permite actuar; y de que la estructura de las alarmas que emite el detector condiciona cómo las veta un modelo capaz (sección 8).

Todo el código, los scripts de procesamiento de datos, los prompts, cada decisión de los modelos con la razón que dieron y todos los resultados están disponibles (sección 9).

2. Antecedentes y trabajo relacionado

Computación autonómica y sistemas autoadaptativos. El modelo de referencia MAPE-K [1] estructura un sistema autogestionado como un bucle de realimentación en torno a un recurso gestionado. La ingeniería de esos bucles es el objeto de la literatura sobre sistemas autoadaptativos [2], que subraya

que la fase Analyze debe separar el cambio real del ruido antes de planificar cualquier adaptación. Recientemente se han estudiado los LLM como componentes de esos bucles: una revisión del estado del arte y hoja de ruta organiza sus usos potenciales en torno a las funciones de MAPE-K [8, 9]; se han integrado agentes basados en LLM en sistemas multiagente basados en MAPE-K [10]; y MAPER extiende MAPE-K con un componente de razonamiento basado en LLM para manejar situaciones imprevistas en tiempo de ejecución [11]. Nuestro estudio se diferencia en que sitúa un LLM dentro del bucle de un sistema industrial cuyo detector heredamos sin cambios, y en que lo mide frente a baselines fuertes sin LLM sobre datos reservados.

Propiedades self- en la fabricación.* Nuestro trabajo previo integró sistemas multiagente, orientación a servicios y computación autonómica para la coordinación de procesos en la Industria 4.0 [3, 4, 12]. El presente artículo reexamina empíricamente esa arquitectura en lugar de proponer una nueva.

Detección de cambios. Detectar un cambio en la tasa de un flujo de eventos es un problema clásico [13]. Nuestros baselines basados en reglas más fuertes son instancias sencillas: un test de Poisson frente a una tasa base estática y un test frente al historial reciente del mismo flujo.

LLM y recuperación en la toma de decisiones industriales. RAG condiciona un modelo de lenguaje con documentos recuperados para cada consulta [7]. En la fabricación, se han combinado agentes LLM con gemelos digitales para planificar y controlar la producción modular [5] y se han aplicado a la planificación y programación avanzadas [6], y una revisión reciente de los LLM para el diagnóstico de fallos en la fabricación inteligente cubre, entre otras funciones, la fusión de conocimiento industrial con recuperación aumentada [14]. Nuestro diseño incluye deliberadamente un baseline estadístico fuerte y reglas de control que aíslan lo que el LLM aporta más allá de ellos.

3. El sistema de 2020, revisado

3.1. Simulador y datos

SSACSimulator es un simulador de eventos discretos en Qt/C++. Un modelo de proceso, derivado del conjunto de datos Bosch Production Line Performance [15], describe 52 estaciones en cuatro sublíneas más los nodos de inicio y fin, 285 transiciones (84 de ellas flujos principales), un tiempo de ciclo de 6 420 min y 1 183 165 casos. Cuatro trazas de eventos de 100 000

transiciones cada una reproducen movimientos de piezas registrados; abarcan entre 54 y 168 horas de producción.

3.2. Port sin interfaz gráfica

Portamos a Python el núcleo de simulación (unas 1 440 líneas de lógica en C++, sin contar la interfaz gráfica). El port reproduce el comportamiento del original tal como se lee en su código fuente, incluidos detalles que cambian los resultados: umbrales calculados en precisión simple, un límite de carga de 100 000 eventos y una regla permisiva para las filas mal formadas. Un camino rápido avanza solo las piezas que pueden salir del sistema; el bucle original por tick se conserva como oráculo de pruebas, y una prueba de regresión comprueba que ambos producen detecciones, decisiones y tiempos de vida de pieza idénticos. Una ejecución tarda entre 0,2 y 0,4 s. La batería de pruebas tiene 87 tests.

3.3. Lo que hace realmente la capa Analyze

El original implementa tres detectores: una estación cuyo tiempo medio de transición supera 1,3 veces su referencia; más del 10 % de las estaciones marcadas dentro de un mismo lote de eventos simultáneos; y un tiempo de vida medio de las piezas que supera 1,1 veces el tiempo de ciclo. En la primera traza solo se dispara el segundo (39 alarmas). El detector de fallos predichos depende de una etiqueta de un clasificador que es positiva en 15 de 400 000 eventos en el conjunto de las trazas (0,004 %). El detector global es inalcanzable: el tiempo de vida medio de las piezas (636 min) está un orden de magnitud por debajo de su umbral (7 062 min). Por último, 45 de las 285 transiciones tienen un tiempo de referencia cero, de modo que cualquier duración positiva marca su estación.

Plan, Execute y Knowledge están ausentes. Ese es el punto de partida del resto del artículo.

4. Cerrar el bucle

La figura 1 resume el bucle cerrado. Los componentes añadidos en este trabajo aparecen sombreados.

4.1. Estaciones gemelas y espacio de acciones

La línea de Bosch está formada por pares de estaciones paralelas (y algunos tríos) que comparten predecesoras y sucesoras y se reparten la carga casi por igual; por ejemplo, en sus flujos principales las estaciones L0_S2

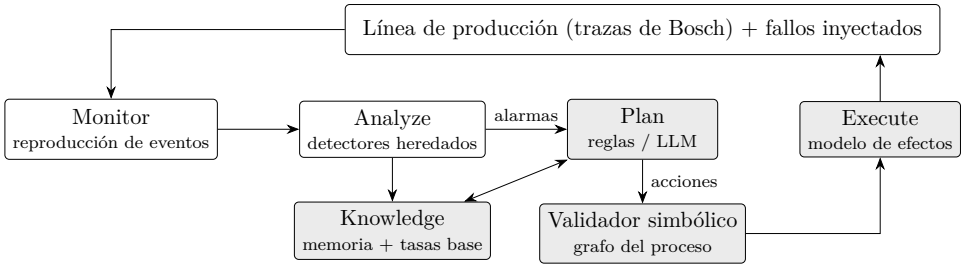


Figura 1: El bucle MAPE-K cerrado. Blanco: presente en el sistema de 2020. Gris: añadido en este trabajo.

y L0_S3 reciben 339 345 y 334 054 casos. Identificamos las gemelas automáticamente a partir del solapamiento de sus conjuntos de predecesoras y sucesoras en los flujos principales: 29 de las 52 estaciones tienen una. Desviar una pieza de una estación a su gemela mantiene coherente su trayectoria, lo que hace del desvío una acción con sentido físico. El espacio de acciones es REROUTE(S, T, f), que desvía una fracción f de las piezas que entran en S hacia su gemela T; RESTORE(S); MAINTENANCE(S, d), que repara S durante d minutos; y NOOP.

4.2. Validador simbólico

Toda acción, de cualquier planificador, se comprueba contra el grafo del proceso y las intervenciones en curso antes de ejecutarse: las estaciones deben existir, el destino de un desvío debe ser una gemela estructural, los desvíos no deben formar cadenas y una estación no puede repararse dos veces a la vez. El validador nunca juzga si una acción es buena idea, solo si es admisible.

4.3. Modelo de efectos

La simulación está guiada por la traza: los instantes de inicio proceden de los datos registrados y no se desplazan; los fallos y las acciones actúan sobre las duraciones de las transiciones, y el tiempo de vida de una pieza es la suma de las duraciones de sus transiciones. Por tanto, el retraso no se propaga en forma de colas; el modelo estima el tiempo perdido por pieza. Una "transición" de Bosch es el tiempo entre dos marcas de estación y mide sobre todo espera: su mediana es 0 min y su percentil 90 es de 18–30 min, pero su percentil 99 llega a 4 100–5 900 min. Por ello los efectos son *aditivos* y proporcionales al tiempo de proceso de cada estación τ_S (la mediana de sus transiciones de hasta 60 min), nunca multiplicativos sobre una transición

que incluye días de almacenamiento. τ_S vale 6 o 12 min en las ocho estaciones donde se inyectan fallos, idéntico en todas las trazas, y como máximo 45–57 min en toda la línea. El modelo de efectos es:

- **Fallo** de factor k en S: $+(k(t)-1)\,\tau_S$ en cada transición que sale de S, donde $k(t)$ es un *escalón* (k desde t_0), *intermitente* (k durante 60 min, 1 durante 120 min, y así sucesivamente) o una *deriva* (crece linealmente de 1 a k en 600 min).
- **Desvío**: una fracción determinista de las piezas (seleccionadas mediante un hash del identificador del caso) va a T, que añade entonces $\alpha\, f\, \tau_T$ por la carga extra.
- **Mantenimiento**: los fallos de S que empezaron antes del final de la reparación se eliminan al terminar esta. Durante la reparación, en modo *contingencia* (por defecto) una capacidad de respaldo fuera del modelo procesa las piezas, añadiendo $0{,}5\,\tau_S$; en modo *parada* las piezas esperan a que termine la reparación. Reparar una estación sana cuesta y no arregla nada, y eso es lo que encarece las falsas alarmas.

El conjunto de datos está anonimizado y no proporciona capacidades, tiempos de reparación ni fallos etiquetados, de modo que todos los fallos son sintéticos y todo parámetro que no pudimos fundamentar en los datos (severidad k, tiempo de reparación d, congestión α, modo de mantenimiento) se varía en un análisis de sensibilidad (sección 7.1).

5. Planificadores

Todos los planificadores reciben, para cada alarma, la alarma y el estado de sus propias intervenciones anteriores. Ninguno ve los fallos inyectados. Todos responden del mismo modo ante una estación que consideran averiada: desviar a una gemela libre si existe y, después, reparar.

5.1. Planificadores basados en reglas

- **Ingenuo**: actúa sobre toda estación nombrada en una alarma.
- **Calibrado**: actúa cuando el número de alarmas de una estación en 120 min es improbable (cola de Poisson, $\alpha = 10^{-3}$) bajo su tasa base, aprendida en una traza sin fallos *distinta*.
- **Adaptativo**: el mismo test, frente a la tasa observada en las 24 h anteriores de la misma traza, de forma causal, tras un calentamiento de 8 h.

- **Adaptativo + veto por carga** (control): la regla adaptativa, ignorando las estaciones que reciben carga que el propio controlador desvió hacia ellas.
- **Solo gemelas** (control): actúa sobre toda estación marcada que tenga una gemela libre e ignora las demás.

Los dos controles no son métodos propuestos; implementan como reglas simples comportamientos observados en planificadores LLM, de modo que la contribución de un LLM se mide más allá de ellos. La ventana del test (120 min), su nivel ($\alpha = 10^{-3}$), el mínimo de alarmas (3), la historia (24 h) y el calentamiento (8 h) se fijaron antes de cualquier evaluación; la sección 7.1 presenta una rejilla sobre los tres primeros. Todos los planificadores, incluidos los LLM, dejan de actuar 240 min sobre una estación en la que acaban de actuar.

5.2. Planificadores LLM

Para cada alarma con estaciones accionables, el LLM debe devolver, para cada una de ellas, un veredicto (*degradada*, *normal*, *autoinducida* o *evidencia insuficiente*) seguido de un tipo de acción (ninguna, reparar, o desviar y reparar). La salida está restringida por un esquema JSON cuyos nombres de estación se limitan a las accionables, de modo que el decodificador no puede inventar una estación. El código elige la gemela, descarta toda acción incoherente con el propio veredicto del modelo y pasa el resto al validador.

- **C1** recibe la alarma, las estaciones accionables, si cada una tiene una gemela libre y las intervenciones en curso: la misma información estructural que las reglas.
- **C2 (RAG)** recibe además conocimiento recuperado de la memoria operativa del controlador: para cada estación, las alarmas observadas en las últimas 2 y 24 h frente al número que predice su tasa base sin fallos, si el controlador le está desviando carga, si sus alarmas bajaron tras reparaciones anteriores y sus registros más recientes. La recuperación se indexa por estación y se ordena por recencia; todas las estadísticas las calcula el código.
- **Híbrido**: el test adaptativo de detección de cambios selecciona las estaciones; el LLM, con el conocimiento de C2, confirma o veta y elige la acción. Una estación vetada no se vuelve a examinar durante 120 min. El prompt indica al modelo que el test suele acertar pero no ve el contexto, y da como ejemplos la carga que el controlador desvió hacia una

estación, una reparación anterior que no redujo sus alarmas y *"a burst of alarms shared by many stations at once"* (una ráfaga de alarmas compartida por muchas estaciones a la vez).

Modelos. Usamos Qwen2.5-7B-Instruct [16], cuantizado a 4 bits y servido localmente con Ollama [17] a temperatura 0; y Claude Fable 5.1 (Anthropic), accedido mediante el Claude Agent SDK [18] con el esfuerzo en *low*, sin herramientas y sin fuentes de contexto externas. El modelo de frontera no expone un ajuste de temperatura, por lo que sus ejecuciones no son reproducibles bit a bit. Todas las peticiones y respuestas de ambos modelos se almacenan, indexadas por la petición completa; las peticiones idénticas, frecuentes porque el tramo de la traza anterior al fallo produce los mismos estados en todos los escenarios, se sirven desde ese almacén (el 78 % de las 7 869 llamadas de C2 sobre Fable 5.1). El almacén nos permite reproducir cada decisión publicada, y el repositorio publica cada decisión con la razón que dio el modelo, pero cada una es una única muestra del modelo: no estimamos la variabilidad entre ejecuciones.

6. Protocolo experimental

Escenarios. Se inyecta un fallo de factor $k = 3\,600$ min después del inicio de una traza en una de ocho estaciones, cuatro con gemela (L0_S2, L0_S4, L0_S6, L3_S35) y cuatro sin ella (L0_S1, L0_S8, L3_S29, L3_S33), todas con tiempos de proceso observables en cada traza, con una de las tres formas de fallo. Cada traza tiene además una ejecución sin fallos. Las reparaciones duran 60 min, $\alpha = 0{,}5$, y el mantenimiento funciona en modo contingencia salvo que se indique lo contrario.

Trazas de desarrollo y reservadas. Los prompts y el contexto recuperado se diseñaron y revisaron solo sobre la primera traza, con cuatro casos de desarrollo (sin fallo, dos fallos en escalón y uno en deriva; ningún fallo intermitente); cada versión y sus resultados quedaron registrados. Durante el desarrollo, las tasas base que recupera C2 y que usa la regla calibrada se aprendieron en la traza 100m–200m, que después queda reservada: esto no expone ninguna etiqueta de fallo, pero una traza reservada sí influyó en la elección de la versión del prompt. Las otras tres trazas están reservadas: sobre ellas los prompts se ejecutaron una vez, congelados. En la evaluación final, las reglas calibradas y C2 aprenden siempre sus tasas base en una traza distinta de aquella en la que se evalúan (100m–200m a partir de la traza de desarrollo; 200m–300m y 500m–600m, una a partir de la otra).

Métricas. Cada ejecución reproduce la misma traza, de modo que cada pieza se compara consigo misma en una ejecución de referencia sin fallos. El *retraso en exceso* de una ejecución es la suma, sobre las piezas, de su tiempo de vida adicional. La *recuperación agregada* de un planificador sobre un conjunto de escenarios es $1 - \sum \text{exceso(planificador)} / \sum \text{exceso(no hacer nada)}$: 1 elimina el coste de los fallos; 0 no es mejor que no hacer nada; los valores negativos significan que las propias acciones del planificador cuestan más que los fallos. El *coste de las falsas alarmas* es el retraso en exceso por hora en las ejecuciones sin fallos. Un *acierto específico* cuenta una acción sobre la estación averiada solo si el mismo planificador, en la misma traza sin el fallo, no actuó sobre esa estación en un margen de $\pm$120 min; excluye las reparaciones de rutina que coinciden por casualidad con el fallo.

Estadística. Tratamos como estimando principal la diferencia de recuperación agregada, con un intervalo bootstrap al 95 % [19] que remuestrea los escenarios dentro de su traza (10 000 remuestreos). Como análisis secundarios: la prueba de rangos con signo de Wilcoxon sobre el retraso en exceso por escenario [20], con ajuste de Holm [21] sobre exactamente las comparaciones de cada bloque de la tabla 3, y la correlación biserial de rangos para muestras pareadas r como tamaño del efecto. El desglose por forma de fallo no se especificó de antemano y es exploratorio. Como los escenarios de una misma traza comparten el ruido de alarmas y tres trazas son pocos grupos, presentamos también un bootstrap en dos niveles que remuestrea trazas y, dentro de ellas, escenarios.

7. Resultados

7.1. Planificadores basados en reglas y sensibilidad

La tabla 1 recoge los planificadores basados en reglas sobre las cuatro trazas (96 escenarios con fallo; las reglas no tienen ningún ajuste específico por traza) y un análisis de sensibilidad de un factor cada vez sobre fallos en escalón. Conectar las alarmas originales directamente a acciones aporta poco: el planificador ingenuo recupera +0,17 en las cuatro trazas y +0,02 en las tres reservadas, y repara 87 estaciones sanas por escenario. Con una pausa de 120 min en lugar de 240 min entre acciones sobre la misma estación es peor que no hacer nada ($-$0,11, 122 reparaciones falsas por escenario). Las reglas calibrada y adaptativa recuperan aproximadamente la mitad del coste de los fallos. El parámetro de congestión α no cambia el signo de ningún resultado. Los fallos leves ($k = 1{,}5$) y las reparaciones largas (240 min) invierten el beneficio de la regla calibrada y reducen el de la adaptativa a

Tabla 1: Planificadores basados en reglas sobre las cuatro trazas: recuperación agregada (escenarios con beneficio neto). Punto central: fallos en escalón, intermitentes y deriva, $k = 3$, reparaciones de 60 min, $\alpha = 0{,}5$, modo contingencia. Filas de sensibilidad: fallos en escalón, un parámetro cambiado. Todos los planificadores hacen una pausa de 240 min entre acciones sobre la misma estación.

	Ingenuo	Calibrado	Adaptativo
Todas las formas de fallo	+0,17 (53/96)	+0,49 (73/96)	+0,56 (69/96)
escalón	+0,35 (24/32)	+0,59 (30/32)	+0,64 (29/32)
intermitente	−0,78 (5/32)	−0,09 (13/32)	+0,07 (13/32)
deriva	+0,33 (24/32)	+0,60 (30/32)	+0,66 (27/32)
Escalón, $\alpha = 0$	–	+0,74	+0,73
Escalón, $\alpha = 1$	–	+0,47	+0,56
Escalón, $k = 1{,}5$	–	−0,39	+0,10
Escalón, $k = 5$	–	+0,75	+0,73
Escalón, reparación de 15 min	–	+0,84	+0,78
Escalón, reparación de 240 min	–	−0,80	+0,12
Escalón, modo parada	–	−1,17	−0,27

+0,10 y +0,12; en modo parada, donde las piezas esperan a que termine la reparación, ambas reglas son peores que no hacer nada. El beneficio del triaje se mantiene, por tanto, con fallos graves en relación con el coste de una reparación y en líneas con capacidad de respaldo durante las reparaciones. El análisis de sensibilidad cubre solo las reglas, no los planificadores LLM. Los fallos intermitentes son la forma más difícil: la mejor regla recupera +0,07.

Reglas reajustadas. Como los hiperparámetros del baseline más fuerte se fijaron de antemano, barrimos su ventana (60–360 min), su nivel de Poisson (10^{-2}–10^{-4}) y su historia (12–48 h), 45 configuraciones, y elegimos una en la traza de desarrollo, igual que los prompts. La configuración elegida (ventana de 60 min, $\alpha = 10^{-2}$) recupera +0,54 en las trazas reservadas con el veto por carga, lo mismo que la configuración por defecto, y +0,06 en los fallos intermitentes. En toda la rejilla, la recuperación en las trazas reservadas va de +0,36 a +0,56; en los fallos intermitentes nunca supera +0,16, ni siquiera eligiendo la configuración en las propias trazas reservadas.

7.2. Comparación en las trazas reservadas

La tabla 2 y la figura 2 recogen todos los planificadores sobre las tres trazas reservadas (72 escenarios con fallo).

Con el modelo de 7B, los dos planificadores LLM que hacen ellos mismos el triaje de las alarmas quedan muy por debajo de las reglas. C1 se comporta

Tabla 2: Trazas reservadas (72 escenarios con fallo, 3 ejecuciones sin fallos). Recuperación: agregada, con intervalo bootstrap estratificado al 95 %. Actúa: escenarios con al menos una reparación de la estación averiada. Aciertos específicos: véase la sección 6. Reparaciones falsas: reparaciones de estaciones sanas por escenario con fallo. Coste de falsas alarmas: min-pieza/h en las ejecuciones sin fallos, media de las tres trazas. Latencia: mediana del tiempo desde el inicio del fallo hasta la primera reparación de la estación averiada, cuando la hay. Reajustada: hiperparámetros elegidos en la traza de desarrollo.

Planificador	Modelo	Recuperación	Actúa	Aciertos específicos	Reparaciones falsas	Coste de falsas alarmas	Latencia (min)
Regla calibrada	–	+0,45 [0,37; 0,51]	72/72	32/72	48,8	108	222
Regla adaptativa	–	+0,52 [0,43; 0,59]	72/72	57/72	32,8	67	3 090
Adaptativa + veto por carga reajustada	–	+0,54 [0,45; 0,61]	72/72	57/72	31,2	64	3 090
	–	+0,54 [0,46; 0,61]	72/72	54/72	32,9	73	3 024
Solo gemelas (control)	–	−0,31 [−0,41; −0,20]	36/72	9/72	65,0	142	150
C1	Qwen 7B	+0,15 [0,03; 0,26]	57/72	22/72	61,6	136	234
C2 (RAG)	Qwen 7B	+0,20 [0,07; 0,30]	72/72	31/72	74,0	159	360
Híbrido	Qwen 7B	+0,41 [0,31; 0,51]	59/72	50/72	24,0	38	3 270
C1	Fable 5.1	0,00 [0,00; 0,00]	0/72	0/72	0,0	0	–
C2 (RAG)	Fable 5.1	**+0,58** [0,51; 0,64]	57/72	39/72	32,8	58	402
Híbrido	Fable 5.1	+0,24 [0,13; 0,35]	34/72	34/72	**5,1**	**7**	4 164

como la heurística de solo gemelas (en la traza de desarrollo hizo el mismo número de reparaciones que el control de solo gemelas, con recuperaciones de +0,80 y +0,78 en un caso y de −0,16 y −0,19 en otro), y el conocimiento recuperado solo lo mejora modestamente. Limitar el LLM a vetar los candidatos de la regla adaptativa (híbrido) reduce aproximadamente un 40 % el coste de las falsas alarmas de la regla adaptativa con veto por carga, pero actúa sobre la estación averiada con menos frecuencia (en 59 de 72 escenarios, frente a 72).

Con el modelo de frontera, la clasificación cambia. El mismo diseño C2 alcanza +0,58, la recuperación agregada más alta de todos los planificadores, pero no porque detecte los fallos más a menudo: actúa sobre la estación averiada en 57 de 72 escenarios, frente a los 72 de las reglas adaptativas, con 39 aciertos específicos frente a 57. Sin conocimiento recuperado (C1), el modelo de frontera no actúa nunca: en sus 8 394 llamadas juzgó todas las estaciones como *evidencia insuficiente*, razonando, por ejemplo, que *"only the alarm list is available; no transition-time data, trends, or reroutes to distinguish true degradation from known false alarms"* (solo dispone de la lista de alarmas, sin tiempos de transición, tendencias ni desvíos para distinguir una degradación real de las falsas alarmas conocidas). Su recuperación es, por tanto, exactamente cero, sin reparaciones falsas. Con la misma información, el modelo de 7B actúa igualmente, con 61,6 reparaciones falsas por escenario. La figura 3 sitúa cada planificador en el compromiso entre la recuperación ante fallos y el coste de actuar sobre el ruido.

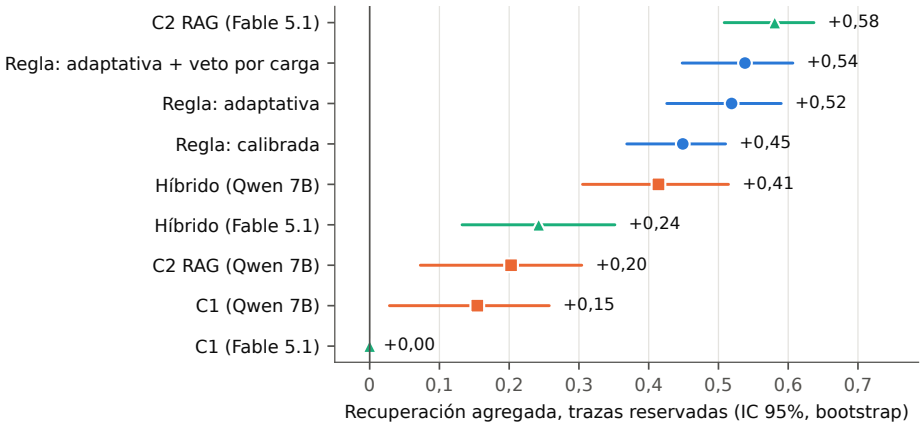


Figura 2: Recuperación agregada en las trazas reservadas. Círculos: planificadores basados en reglas; cuadrados: Qwen2.5 7B; triángulos: Claude Fable 5.1.

7.3. Comparación estadística

La tabla 3 compara C2 sobre Fable 5.1 con los demás planificadores. Frente a la mejor regla, la diferencia de recuperación agregada es de +0,04, con un intervalo al 95 % de −0,01 a +0,10; remuestreando trazas completas se amplía a [−0,05; +0,11]. Por escenario, C2 sobre Fable 5.1 pierde menos tiempo de producción en 50 de 72 escenarios ($p = 0{,}001$, $r = 0{,}43$), pero esos escenarios no son independientes y la ganancia es desigual entre trazas: va por delante en las trazas 100m–200m (+0,09) y 500m–600m (+0,04) y por detrás en la 200m–300m (−0,02). Frente a la regla reajustada, la diferencia es de +0,04 [−0,02; +0,09], mejor en 48 de 72 escenarios. Leemos el resultado global como un empate, no como una victoria.

Por forma de fallo (figura 4), los dos planificadores son indistinguibles en los fallos en escalón y deriva. En los fallos intermitentes, C2 sobre Fable 5.1 es mejor en 21 de 24 escenarios, con $\Delta = +0{,}21$, pero el intervalo llega al cero ([0,00; +0,42]; [−0,14; +0,47] al remuestrear trazas), el 82 % de los minutos-pieza que ahorra en estos fallos procede de la traza 100m–200m, y frente a la regla reajustada la diferencia baja a +0,13 [−0,04; +0,30]. Ninguna configuración de la rejilla de reglas alcanza su +0,19 en los fallos intermitentes, de modo que la dirección es consistente, pero la evidencia no establece una ventaja. Tampoco se trata de una mejor detección: en los fallos intermitentes las reglas adaptativas actúan sobre la estación averiada en los 24 escenarios, y C2 sobre Fable 5.1 en 18.

El momento de actuar explica la mayor parte de la diferencia agregada.

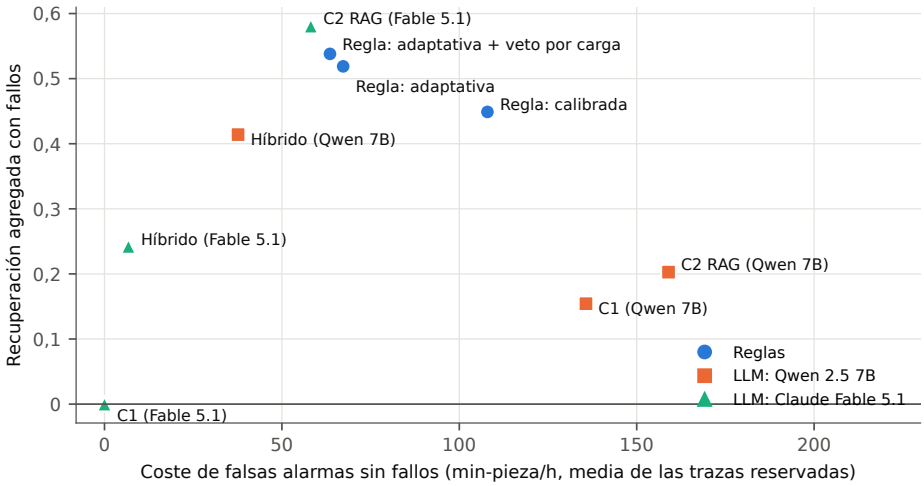


Figura 3: Recuperación con fallos frente al coste de las falsas alarmas sin fallos, trazas reservadas.

Las reglas adaptativas reparan por primera vez la estación averiada una mediana de 3 090 min (unos dos días) después del inicio del fallo; C2 sobre Fable 5.1, cuando actúa, a los 402 min. En los 57 escenarios en que ambos actúan, la diferencia pareada mediana es de 0 min ($p = 0{,}17$): la mayoría de las diferencias son de una o dos horas, pero en 15 escenarios Fable 5.1 actúa más de 1 000 min antes, y esos 15 explican el 96 % de los minutos-pieza netos que ahorra. Actuar pronto no es exclusivo del LLM: la regla adaptativa reajustada sin veto por carga actúa a los 384 min de mediana y alcanza la misma recuperación agregada (+0,54). Como la recuperación se mide sobre toda la traza (54–168 h), una primera reparación tardía de un fallo persistente sigue recuperando buena parte de su coste; una planta que necesite respuesta dentro del turno daría a la latencia de dos días de la regla por defecto más peso del que le da esta métrica.

7.4. El híbrido y el artefacto de las alarmas agrupadas

El híbrido se comporta al revés con el modelo de frontera: se vuelve extremadamente conservador (5,1 reparaciones falsas por escenario, coste de falsas alarmas de 7 min-pieza/h) y actúa sobre la estación averiada solo en 34 de 72 escenarios. Fable 5.1 vetó al menos una estación en 1 256 de sus 1 437 llamadas. Codificados con una regla explícita de palabras clave (*burst*, *line-wide*, *common cause*, *systemic*; la regla y los recuentos están en el repositorio), el 98 % de esos vetos, y los 167 vetos de una estación que

Tabla 3: C2 (RAG) sobre Claude Fable 5.1 frente a otros planificadores, pareado sobre los escenarios con fallo de las trazas reservadas. Valores p de Wilcoxon ajustados por Holm sobre las filas de cada bloque; r: correlación biserial de rangos para muestras pareadas. Δ: diferencia de recuperación agregada (positiva a favor de C2 sobre Fable 5.1), intervalo bootstrap al 95 % estratificado por traza. El desglose por forma de fallo es exploratorio.

Frente a	Mejor en	p (Holm)	r	Δ recuperación [IC 95 %]
Adaptativa + veto por carga	50/72	0,001	0,43	+0,04 [−0,01; +0,10]
Regla adaptativa	53/72	$< 0{,}001$	0,53	+0,06 [+0,01; +0,13]
Regla calibrada	63/72	$< 0{,}001$	0,64	+0,13 [+0,09; +0,18]
C2 (RAG), Qwen 7B	69/72	$< 0{,}001$	0,99	+0,38 [+0,31; +0,47]
C1, Fable 5.1	55/72	$< 0{,}001$	0,75	+0,58 [+0,51; +0,64]
Frente a reglas reajustadas en la traza de desarrollo				
Adaptativa + veto por carga	48/72	0,022	0,31	+0,04 [−0,02; +0,09]
Regla adaptativa	43/72	0,016	0,36	+0,04 [−0,01; +0,10]
Frente a adaptativa + veto por carga, por forma de fallo				
escalón	14/24	0,46	0,23	+0,01 [−0,06; +0,10]
intermitente	21/24	0,003	0,73	+0,21 [0,00; +0,42]
deriva	15/24	0,46	0,29	+0,02 [−0,06; +0,10]

realmente estaba averiada, argumentan que las alarmas llegan en ráfagas compartidas por muchas estaciones y, por tanto, apuntan a una causa común de toda la línea. Por ejemplo, con L0_S4 averiada: *"Every alarm is a line-wide burst (all L0 stations plus L3) with twins alarming together; points to a common upstream cause, not individual faults. Hold off."* ("Cada alarma es una ráfaga de toda la línea (todas las estaciones L0 más L3), con las gemelas alarmando juntas; apunta a una causa común aguas arriba, no a fallos individuales. Esperar.") La premisa es un artefacto del detector: emite una alarma *solo* cuando más del 10 % de las estaciones se marcan en el mismo lote, de modo que toda alarma que produce es, por construcción, una ráfaga compartida. Sin embargo, el prompt del híbrido menciona esas ráfagas entre los contextos que pueden justificar dudar del test, y el prompt de sistema describe la regla del lote. Con el mismo prompt, el modelo de 7B usa este argumento en el 1 % de sus vetos (el 33 % menciona alarmas compartidas de algún modo, a menudo concluyendo aun así que la estación está degradada). Las razones declaradas son, por tanto, compatibles con un modelo capaz que aplica una pista que le dimos a una evidencia cuya estructura impone el detector. No muestran que el modelo descubriera el artefacto, ni separan el razonamiento de un seguimiento más fiel de las instrucciones; para identificar el mecanismo haría falta una ejecución sin esa pista.

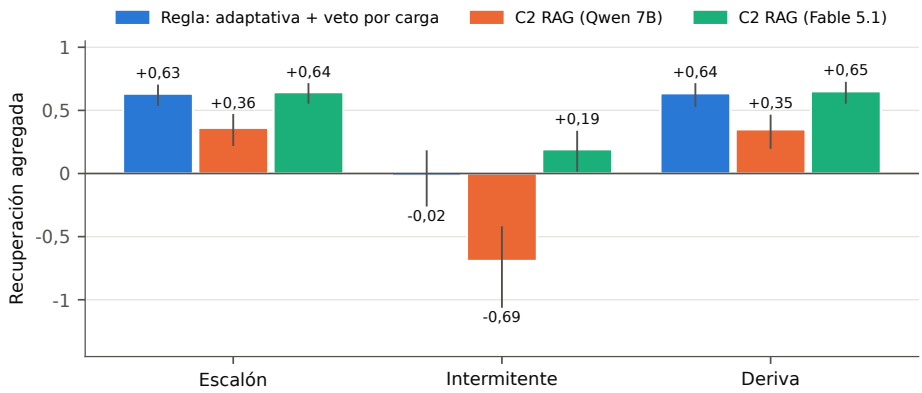


Figura 4: Recuperación agregada por forma de fallo en las trazas reservadas (24 escenarios por forma). Bigotes: intervalo bootstrap al 95 % estratificado por traza.

8. Discusión

El cuello de botella es el triaje, no la planificación. Una vez que una estación se juzga correctamente como averiada, basta una regla sencilla (desviar a una gemela libre, reparar) en nuestro espacio de acciones. Las diferencias que medimos entre planificadores provienen de decidir qué alarmas merecen una acción, y cuándo. Cerrar el bucle del sistema de 2020 sin triaje habría reparado decenas de estaciones sanas por cada fallo sin ninguna ganancia neta.

Un modelo de frontera iguala a un buen test estadístico; uno de 7B, no. El mismo diseño con recuperación aumentada pasa de +0,20 a +0,58 al sustituir el modelo de 7B por el de frontera. Es un contraste entre dos sistemas, que difieren en tamaño, cuantización, proveedor y condiciones de inferencia, no una curva de capacidad medida. Los dos modelos también usan la recuperación de forma distinta. Sin ella, el modelo de 7B actúa sobre alarmas que no puede evaluar, y la recuperación solo lo mejora de +0,15 a +0,20; el modelo de frontera se abstiene, y la recuperación es lo que convierte esa abstención en +0,58. Para un modelo capaz, el componente Knowledge no es un refinamiento opcional, sino la condición para actuar. En el mejor de los casos, el planificador LLM iguala a un test de detección de cambios bien ajustado, que es más barato, más rápido de evaluar y totalmente reproducible. En los fallos intermitentes, cuyo patrón de encendido y apagado diluye las tasas de alarma, queda por delante de todas las configuraciones de reglas

que probamos; es una dirección que merece probarse con más trazas, no una afirmación de superioridad.

La interfaz Analyze–Knowledge condiciona al planificador. Como veto sobre el detector heredado, el modelo de frontera fue menos útil que el de 7B, y sus razones se apoyan en la estructura agrupada de las alarmas. Sea cual sea el mecanismo exacto, en términos de MAPE-K la información que pasa de Analyze a Plan (alarmas agrupadas) no lleva la evidencia por estación que un planificador necesita para distinguir una estación degradada de una perturbación de toda la línea. La implicación para la arquitectura es pasar evidencia por estación en lugar de alarmas agrupadas, y probar ese cambio antes de invertir en planificadores más capaces.

Implicaciones para la práctica. Dentro de los límites de un estudio con fallos sintéticos, tres ideas son trasladables a arquitectos de automatización e ingenieros de mantenimiento. Primero, conectar alarmas por umbral directamente a órdenes de mantenimiento es, en el mejor caso, inútil y, en el peor, perjudicial, porque las reparaciones falsas se acumulan; las alarmas deben racionalizarse antes de que disparen acciones, como ya exige la práctica de gestión de alarmas para los operadores [22]. Segundo, un test de cambio de tasa sobre el historial reciente de cada estación es una capa de triaje barata y transparente, y la referencia con la que debería compararse cualquier controlador aprendido o basado en LLM. Tercero, su ventana es un parámetro de diseño: una ventana corta responde dentro del turno con la misma recuperación. Un planificador LLM puede rendir al mismo nivel, pero su no determinismo y su sensibilidad a cómo se le presentan las alarmas aconsejan mantenerlo como asesor, con toda acción pasando por un validador simbólico, como aquí, y, en una planta, por la aprobación de una persona.

Los conjuntos de desarrollo engañan. En la traza de desarrollo, tres conclusiones fueron erróneas: el planificador de 7B con recuperación aumentada parecía superar a la regla calibrada, el híbrido parecía igualar a su control con un coste de falsas alarmas menor y el modelo de frontera parecía equivalente al de 7B. Solo las trazas reservadas las invirtieron. Los casos de desarrollo no incluían ningún fallo intermitente, la forma en la que más difieren los resultados en las trazas reservadas. Una sola traza con cuatro casos es demasiado pequeña para seleccionar diseños, y los controladores basados en LLM deberían evaluarse con datos que no se usaron para escribir sus prompts.

Amenazas a la validez. De constructo: los fallos son sintéticos y el modelo de efectos es aditivo y guiado por la traza, sin colas, de modo que toda recuperación es una propiedad del modelo, no una propiedad medida de la línea de Bosch. Sus parámetros no verificables se barrieron de uno en uno, para las reglas y solo con fallos en escalón: el parámetro de congestión no cambia ninguna conclusión, pero los fallos leves, las reparaciones largas y el modo parada reducen o invierten el beneficio del triaje, y no se hizo análisis de sensibilidad para los planificadores LLM. La recuperación se mide en tiempo de producción sobre toda la traza, sin ponderación económica, y por eso valora una reparación tardía de un fallo persistente. *Interna*: la métrica de acierto específico depende de una ventana de ±120 min (con ±60 y ±240 min, C2 sobre Fable 5.1 obtiene 44 y 37 de 72, y las reglas adaptativas 57 y 57); cada decisión del modelo de frontera es una única muestra almacenada con esfuerzo bajo, y el 78 % de sus llamadas reutiliza una respuesta almacenada; los prompts forman parte del tratamiento, y el prompt del híbrido contiene la pista que citan sus vetos; el modelo de 7B está cuantizado a 4 bits. El modelo de frontera puede haber visto nuestros artículos de 2020, que nombran la línea de Bosch, pero los fallos son sintéticos y no están descritos en ningún sitio que haya podido ver. *Externa*: una sola línea de producción, un solo conjunto de datos y un solo detector heredado; dos modelos de lenguaje, sin tamaños intermedios. *Estadística*: los escenarios de una misma traza no son independientes; tres trazas reservadas son pocos grupos, y el bootstrap por trazas da intervalos más anchos que el estratificado.

9. Conclusión

Cerramos el bucle autonómico de una arquitectura de fabricación autocoordinada que, tal como se publicó, solo detectaba. Hacerlo de forma ingenua no aporta ningún beneficio, porque el problema decisivo es el triaje de alarmas. Un test clásico de detección adaptativa de cambios es un baseline sólido, y reajustarlo no cambia eso. Un LLM de frontera con conocimiento operativo recuperado lo iguala, mientras que el mismo diseño sobre un modelo de 7B no lo consigue; en los fallos intermitentes queda por delante de todas las reglas que probamos, un resultado exploratorio que necesita más trazas. Usado como veto sobre alarmas agrupadas, el modelo de frontera se vuelve demasiado conservador y cita la estructura de toda la línea que el detector impone a cada alarma. El trabajo futuro rediseñará la interfaz Analyze–Knowledge para pasar evidencia por estación, ejecutará el híbrido sin la pista de las ráfagas, añadirá tamaños de modelo intermedios y

muestras repetidas, y validará el modelo de efectos con datos de planta con capacidades y tiempos de reparación reales.

Disponibilidad de datos y código

El simulador sin interfaz gráfica, los planificadores, los prompts, el registro de desarrollo, cada decisión de los modelos con la razón que dieron, los resultados y los scripts que producen cada tabla y figura están disponibles en `https://github.com/mbsanchez/ssacsimulator`. El simulador usa solo la biblioteca estándar de Python; la estadística y las figuras usan además SciPy y matplotlib. El conjunto de datos de Bosch es público [15]; los ficheros derivados de él (modelo de proceso y trazas de eventos) no se redistribuyen, y el repositorio incluye los scripts que los producen y sus sumas de verificación.

Declaración de contribución de autoría (CRediT)

Manuel Sánchez: conceptualización, metodología, investigación, visualización, redacción – borrador original. **Ernesto Exposito:** supervisión, redacción – revisión y edición.

Declaración de conflicto de intereses

Los autores declaran no tener intereses financieros en conflicto ni relaciones personales conocidas que pudieran haber influido en el trabajo presentado en este artículo.

Declaración sobre el uso de IA generativa y tecnologías asistidas por IA en la preparación del manuscrito

Durante la preparación de este trabajo los autores usaron Claude (Anthropic) para asistir en el desarrollo del software de simulación y análisis, redactar y revisar el texto del manuscrito, y como agente adversario que actuó como revisor para identificar debilidades y sugerir mejoras. El software fue revisado por el primer autor. Tras usar esta herramienta, los autores revisaron y editaron el contenido según fue necesario y asumen toda la responsabilidad sobre el contenido del artículo publicado. El uso de Qwen2.5 y Claude Fable 5.1 como planificadores forma parte del método estudiado en este artículo (sección 5) y no guarda relación con la preparación del manuscrito.

Referencias

[1] J. O. Kephart, D. M. Chess, The vision of autonomic computing, Computer 36 (1) (2003) 41–50. doi:10.1109/MC.2003.1160055.

[2] D. Weyns, An Introduction to Self-adaptive Systems: A Contemporary Software Engineering Perspective, Wiley-IEEE Press, 2020.

[3] M. Sánchez, E. Exposito, J. Aguilar, Implementing self-* autonomic properties in self-coordinated manufacturing processes for the Industry 4.0 context, Computers in Industry 121 (2020) 103247. doi:10.1016/j.compind.2020.103247.

[4] M. Sánchez, E. Exposito, J. Aguilar, Autonomic computing in manufacturing process coordination in Industry 4.0 context, Journal of Industrial Information Integration 19 (2020) 100159. doi:10.1016/j.jii.2020.100159.

[5] Y. Xia, M. Shenoy, N. Jazdi, M. Weyrich, Towards autonomous system: flexible modular production system enhanced with large language model agents, in: 2023 IEEE 28th International Conference on Emerging Technologies and Factory Automation (ETFA), 2023, pp. 1–8. doi:10.1109/ETFA54631.2023.10275362.

[6] M. Li, Q. Zhou, W. Li, T. Qu, M. Yang, P. Jiang, A4PS: Agentic AI-assisted advanced planning and scheduling with large language models for smart manufacturing, Journal of Manufacturing Systems 85 (2026) 207–226. doi:10.1016/j.jmsy.2026.01.003.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, D. Kiela, Retrieval-augmented generation for knowledge-intensive NLP tasks, in: Advances in Neural Information Processing Systems, Vol. 33, 2020, pp. 9459–9474.

[8] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Generative AI for self-adaptive systems: State of the art and research roadmap, ACM Transactions on Autonomous and Adaptive Systems 19 (3) (2024) 1–60. doi:10.1145/3686803.

[9] J. Li, M. Zhang, N. Li, D. Weyns, Z. Jin, K. Tei, Exploring the potential of large language models in self-adaptive systems, in: Proceedings of the 19th International Symposium on Software Engineering for Adaptive

and Self-Managing Systems (SEAMS), 2024, pp. 77–83. doi:10.1145/3643915.3644088.

[10] N. Nascimento, P. Alencar, D. Cowan, Self-adaptive large language model (LLM)-based multiagent systems, in: 2023 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C), 2023, pp. 104–109. doi:10.1109/ACSOS-C58168.2023.00048.

[11] L. Alves, G. L. B. De Oliveira, M. Chagas, A. Bandeira, P. H. Maia, C. Rocha, MAPER: Extending MAPE-K with LLM-based reasoning to manage unanticipated situations in self-adaptive systems, in: Proceedings of the 21st International Conference on Software Engineering for Adaptive and Self-Managing Systems (SEAMS), 2026, pp. 239–250. doi:10.1145/3788550.3794886.

[12] M. Sánchez, E. Exposito, J. Aguilar, Industry 4.0: survey from a system integration perspective, International Journal of Computer Integrated Manufacturing 33 (10–11) (2020) 1017–1041. doi:10.1080/0951192X.2020.1775295.

[13] M. Basseville, I. V. Nikiforov, Detection of Abrupt Changes: Theory and Application, Prentice Hall, 1993.

[14] N. D. Thuan, L. H. Anh, Large language models for fault diagnosis in smart manufacturing: A comprehensive review, Intelligent Systems with Applications 31 (2026) 200714. doi:10.1016/j.iswa.2026.200714.

[15] Bosch, Bosch production line performance, Kaggle competition dataset, https://www.kaggle.com/c/bosch-production-line-performance (2016).

[16] Qwen Team, Qwen2.5 technical report, arXiv:2412.15115 (2024).

[17] Ollama, Ollama: run large language models locally, https://ollama.com, version 0.35.1 (2026).

[18] Anthropic, Claude agent sdk, https://code.claude.com/docs/en/agent-sdk, python package claude-agent-sdk 0.2.163 (2026).

[19] B. Efron, R. J. Tibshirani, An Introduction to the Bootstrap, Chapman & Hall, 1993.

[20] F. Wilcoxon, Individual comparisons by ranking methods, Biometrics Bulletin 1 (6) (1945) 80–83.

[21] S. Holm, A simple sequentially rejective multiple test procedure, Scandinavian Journal of Statistics 6 (2) (1979) 65–70.

[22] International Society of Automation, ANSI/ISA-18.2-2016: Management of alarm systems for the process industries, Tech. rep., International Society of Automation, Research Triangle Park, NC (2016).